

Dimensionality Reduction & Clustering

Dr. Krishna Bathula

Learning Objectives

1. Unsupervised Learning
2. Dimensionality Reduction
3. Principle Component Analysis
4. Clustering
5. K-Means Clustering
6. DBSCAN
7. Perceptron Learning Algorithm
8. Multi-Layer Perceptron

Unsupervised Learning

Unsupervised learning is a type of machine learning where the model learns **patterns and structures** from input data **without explicit supervision** or labeled responses.

In unsupervised learning, the algorithm tries to find **hidden structures in unlabeled data**. There is **no output**.

This can include tasks such as **clustering**, where the algorithm **groups similar data points together**, or **dimensionality reduction**, where the algorithm **reduces the number of features** in the data while preserving its important characteristics.

Unsupervised learning is often used for exploratory data analysis and finding patterns in data where labeled examples are not available.

Curse of Dimensionality

Many Machine Learning problems involve thousands or even millions of features for each training instance.

Not only does this make training extremely slow, it can also make it much harder to find a good solution, as we will see.

This problem is often referred to as the curse of dimensionality.

It is often possible to reduce the number of features considerably, turning an intractable problem into a tractable one

Example - Curse of Dimensionality

House Prices Prediction problem

Model-1

1. Price
2. Area of House

Accuracy ~ 72%

Model-2

1. Price
2. Area of House
3. Age

Accuracy ~ 84%

Model-3

1. Price
2. Area of House
3. Age
4. Location

Accuracy ~ 96%

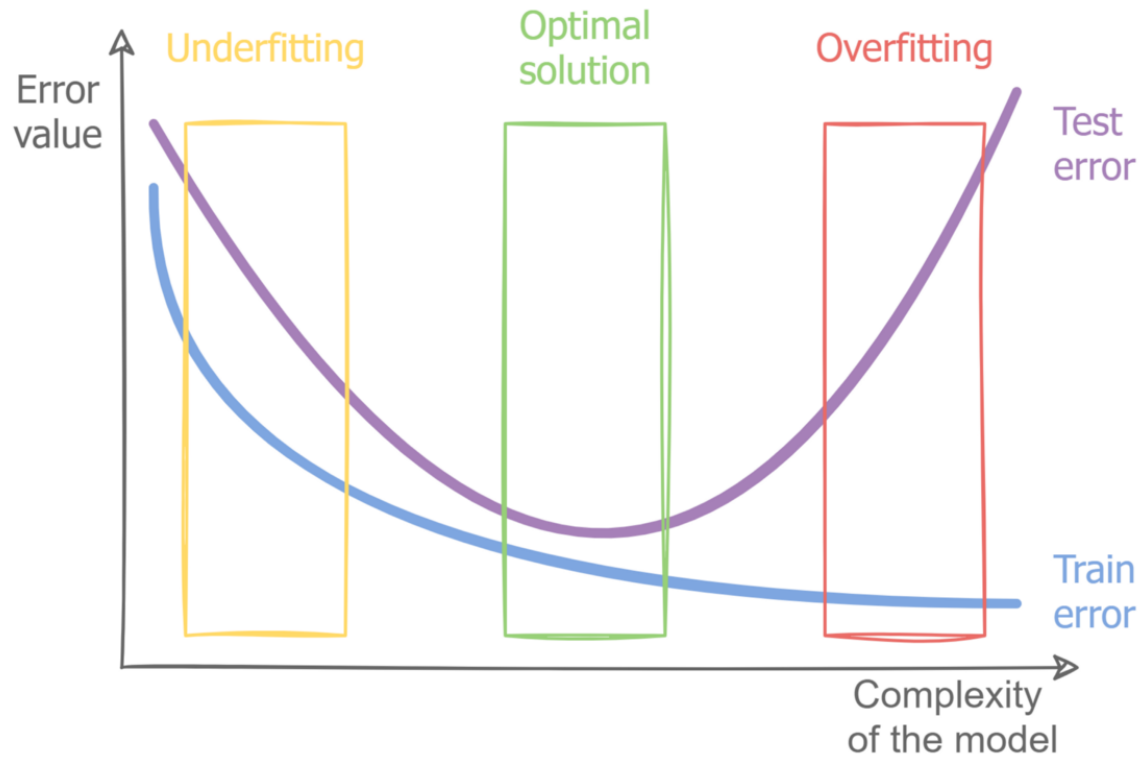
Model-4

1. Price
2. Area of House
3. Age
4. Location
5. .
6. .
7. .
8. .

Accuracy ~ 

Overfitting

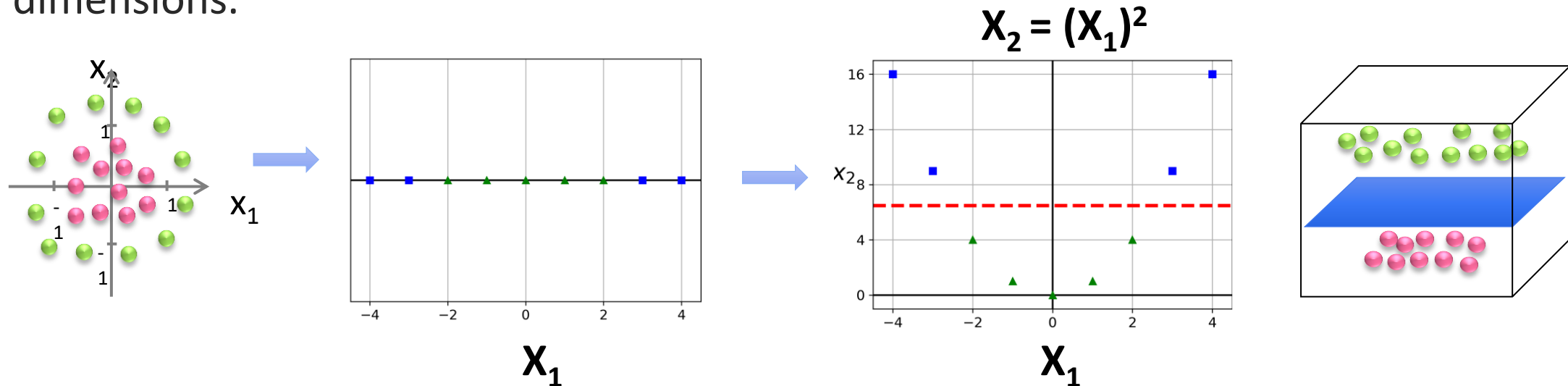
Curse of Dimensionality



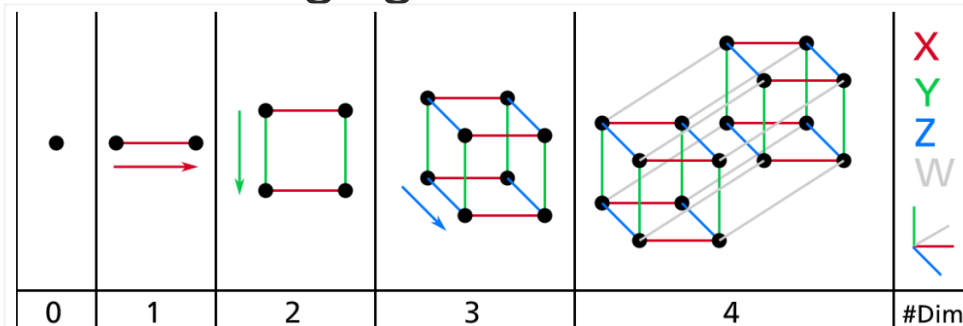
Scalar → Vectors → Matrix → Tensors

Curse of Dimensionality

For non-linear datasets, we projected the 2-dimensional data points into 3 dimensions.



As the dimensions increase, it is more challenging to visualize and train the model. This is not always the best option.



Avoid Curse of Dimensionality

The number of training instances required to reach a given density grows exponentially with the number of dimensions.

To improve the Model's performance, we have to prevent the curse of dimensionality.

There are two main approaches for dimensionality reduction:

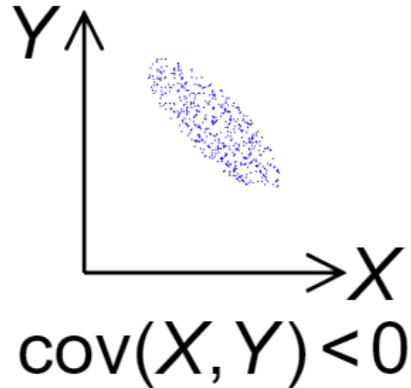
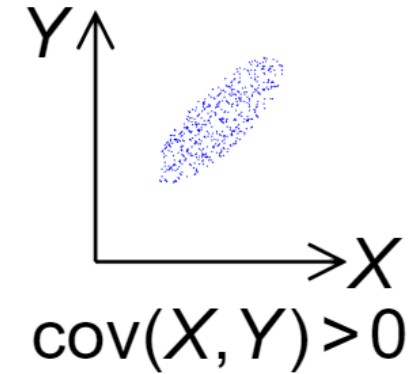
1. **Feature Selection:** Selection of features that are very important to the model.
2. **Dimensionality Reduction:** (PCA) We opt for feature extraction in this process.

Feature Engineering is applicable.

Feature Selection - Covariance

1. X increases y increases (linear). X decreases, y decreases.

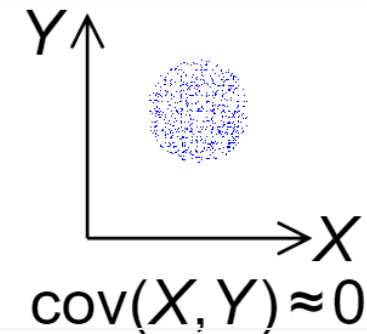
Positive relationship



2. X increases y decreases. X decreases y increases.

Negative relationship

3. X increases Y increases linear. **No relationship**



Feature Selection - Covariance

Population Covariance Formula

$$\text{Cov}(x, y) = \frac{\sum (x_i - \bar{x})(y_i - \bar{y})}{N}$$

Sample Covariance

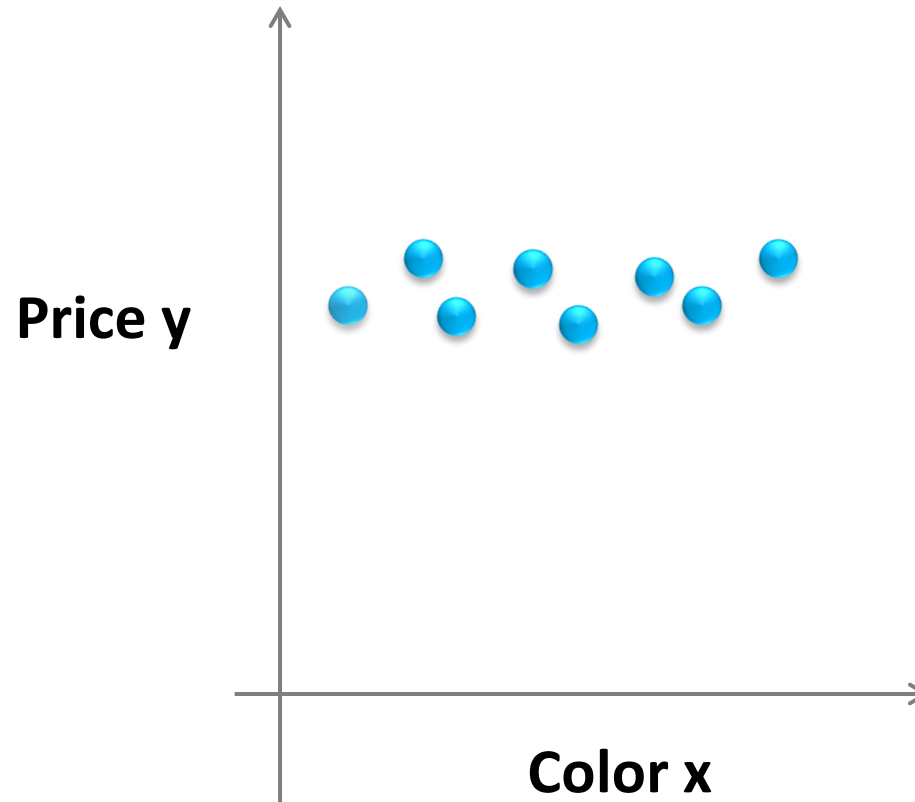
$$\text{Cov}(x, y) = \frac{\sum (x_i - \bar{x})(y_i - \bar{y})}{N-1}$$

For a positive covariance value, we will have a **positive relationship** among the variables. If it approximates **0**, then there is **no relationship**.

High positive covariance indicates that the feature is most important.

Feature Selection - Covariance

Example: The **color** of the house vs **house price**



Feature Creation

Example: The no. of floors, no. of bed-rooms, no. of bathrooms ... very important.

Reduce dimensions by limiting the features...

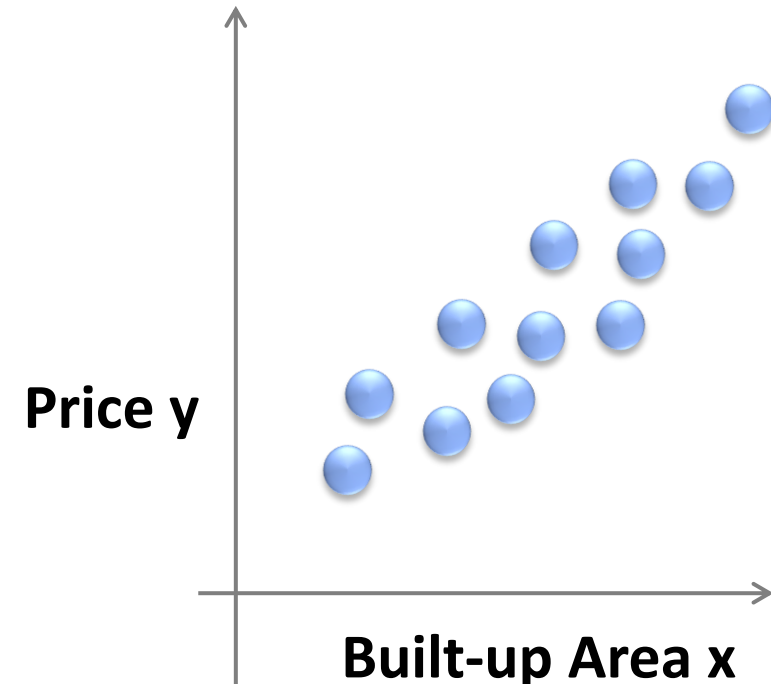
Create new feature built-up area based on the above 3 features

This is possible with the help of SME.

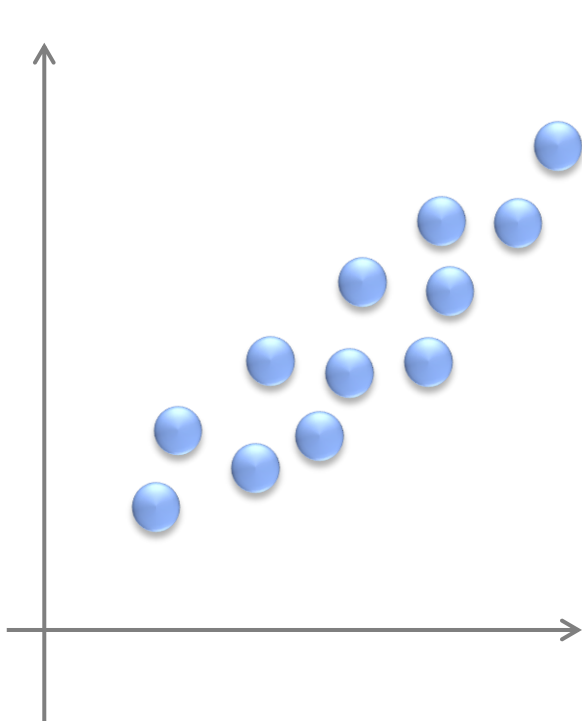
Here we are reducing three dimensions to a single.

Some information is lost (no. floors...).

This problem is handled using PCA.



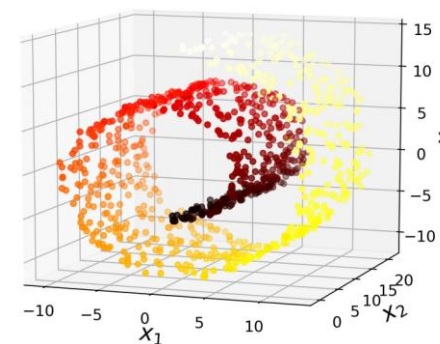
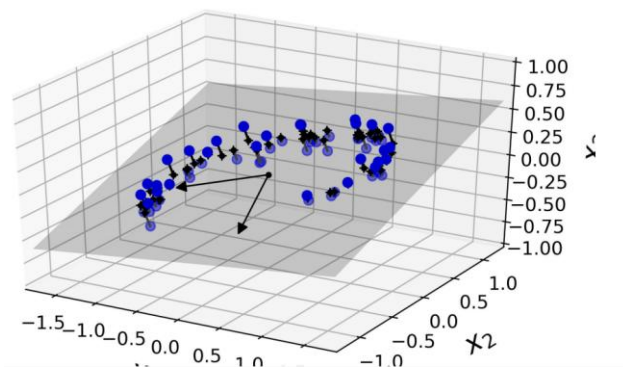
Projection



Projection

To handle the **loss of information**, project every training instance perpendicularly onto this subspace and to get the new 2D dataset.

However, projection is not always the best approach to dimensionality reduction. In many cases the subspace may twist and turn, such as a Swiss roll.



In short, if you reduce the dimensionality of your training set before training a model, it will usually speed up training, but it may not always lead to a better or simpler solution as it all depends on the dataset.

PCA

Principal Component Analysis (PCA)

Principal Component Analysis (PCA) is a statistical method used to **simplify complex data** by reducing the **number of variables** while retaining as much of the original information as possible.

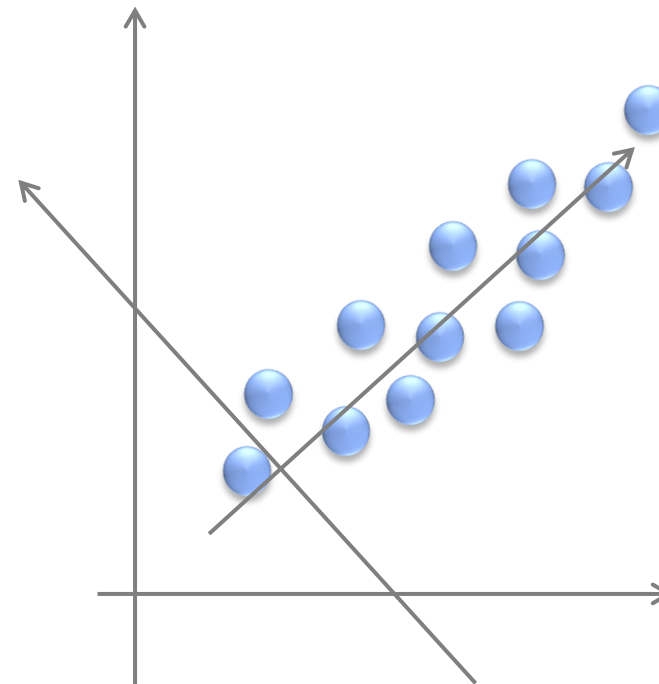
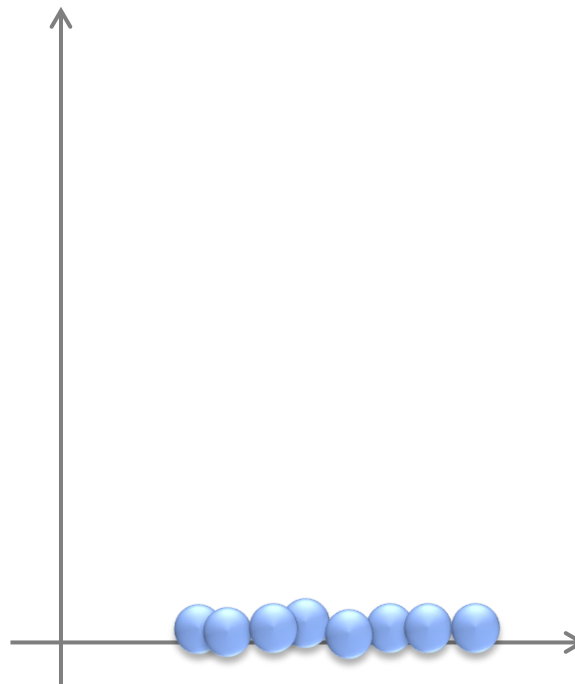
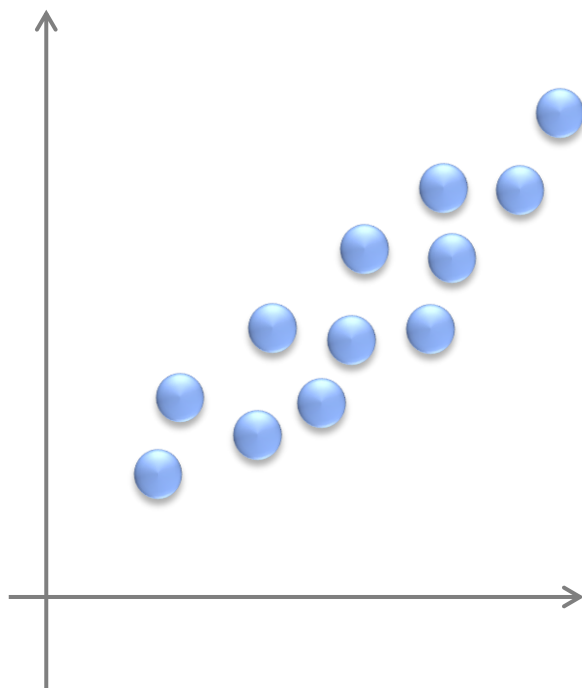
The method achieves this by transforming the data into a **new set of variables** called **principal components**, which are linear combinations of the original variables.

The first principal component explains the most **variance in the data**, and each **subsequent component** explains **as much variance as possible** while being **uncorrelated** with the previous components.

This allows for a reduction in the dimensionality of the data while still retaining as much information as possible.

Projection

Eigen decomposition



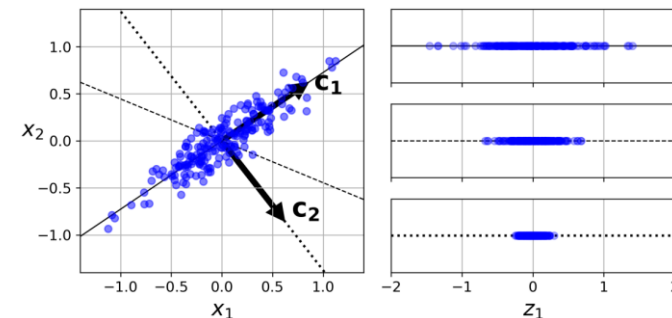
Principle Components

Finding the Principal Components of a training set...

PCA identifies the axis that accounts for the largest amount of variance in the training set.

It also finds a second axis, orthogonal to the first one, that accounts for the largest amount of remaining variance.

If it were a higher-dimensional data-set, PCA would also find a third axis, orthogonal to both previous axes, and a fourth, a fifth, and so on—as many axes as the number of dimensions in the dataset.



PCA Steps

The PCA process involves several steps.

First, the data is standardized by centering and scaling it to have a mean of zero and a standard deviation of one. This ensures that each variable contributes equally to the analysis.

Next, the covariance matrix of the standardized data is calculated.

The **eigenvectors** and **eigenvalues** of this matrix are then calculated. (The eigenvectors are used as the principal components)

The **eigenvalues** represent the amount of variance explained by each **principal component**.

Singular Value Decomposition

How to Find the Principal Components of a training set...

There is a standard matrix factorization technique called Singular Value Decomposition (SVD).

This can decompose the training set matrix X into the matrix multiplication of three matrices $\mathbf{U} \mathbf{\Sigma} \mathbf{V}^T$, where $\mathbf{V}$ contains all the principal components that we are looking for.

$$\mathbf{V} = \begin{pmatrix} | & | & & | \\ \mathbf{c}_1 & \mathbf{c}_2 & \cdots & \mathbf{c}_n \\ | & | & & | \end{pmatrix}$$

PCA Advantages and Disadvantages

PCA can be applied to a variety of data types, including numerical and categorical data.

It is commonly used in fields such as finance, genetics, and image processing, among others.

PCA can also be modified in various ways, such as kernel PCA and incremental PCA, to better suit different types of data or situations.

PCA - Limitations.

It assumes that the data is linearly correlated, and it may not perform well when this assumption is violated.

The interpretation of the PCA may not always be straightforward.

Linear Discriminant Analysis (LDA)

LDA is a supervised dimensionality reduction technique that aims to **maximize the separation between different classes** in the data while reducing the dimensionality.

It achieves this by **finding a set of axes** that maximize the **between-class scatter** and minimize the **within-class scatter**.

LDA **computes the eigenvectors** of the **between-class scatter matrix** and uses them as the **new axes for projection**.

It then **projects the data onto these axes** to obtain the transformed features.

Linear Discriminant Analysis (LDA)

LDA is commonly used for **feature extraction** in classification tasks, where the objective is to find **discriminative features** that can distinguish between different classes.

It can improve the performance of **classification algorithms** by reducing the dimensionality of the input space.

The downside is that LDA assumes that the **data is normally distributed** and that the **classes have equal covariance matrices**.

It may not perform well if these **assumptions are violated** or if the **classes are highly overlapping**.

t-SNE

t-distributed Stochastic Neighbor Embedding (t-SNE)

t-SNE is a non-linear dimensionality reduction technique that is particularly effective for **visualizing high-dimensional data** in two or three dimensions while **preserving the local structure** of the data.

It aims to represent **each data point as a point in a low-dimensional space** such that similar points are modeled as nearby and dissimilar points are modeled as far apart.

t-SNE **constructs a probability distribution over pairs of data points** in the high-dimensional space and a **probability distribution over pairs of points in the low-dimensional space**.

It then **minimizes the Kullback-Leibler divergence** between these two distributions using gradient descent.

t-SNE

t-SNE is commonly used for visualizing high-dimensional datasets in a low-dimensional space, particularly in fields such as image processing, natural language processing, and genomics.

It can reveal underlying structures, clusters, and relationships in the data that may not be apparent in the original space.

Downsides:

t-SNE is computationally expensive, especially for large datasets, and the results may vary depending on the choice of hyperparameters such as the perplexity and the number of iterations.

It is also sensitive to the choice of distance metric and may produce different embeddings for different runs of the algorithm.

Clustering

Clustering is an unsupervised learning technique used to group similar data points based on their characteristics or attributes.

The goal of clustering is to partition a dataset into subsets, or clusters, such that data points within the same cluster are more similar to each other than they are to data points in other clusters.

In clustering, there is no predefined notion of "correct" labels or categories for the data.

These algorithms identify patterns or structures in the data based on similarity measures or distance metrics. Common clustering algorithms include K-means, hierarchical clustering, DBSCAN, and Gaussian mixture models.

K Means

K-Means Clustering

k-means clustering is a method of vector quantization.

K-Means Clustering is based on centroids and based on Euclidean distance.

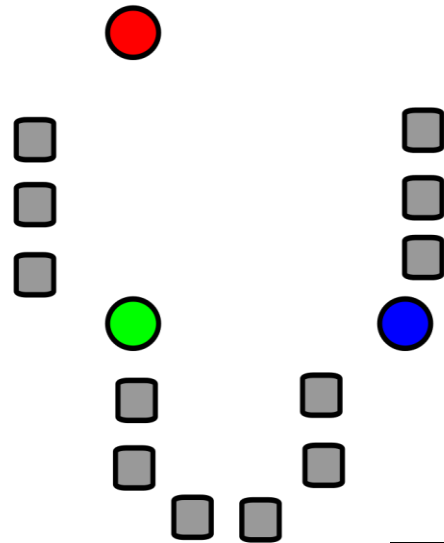
We try with different K-value so that we get different centroids

First, initialize with $k=2$. Initialize the centroids randomly.

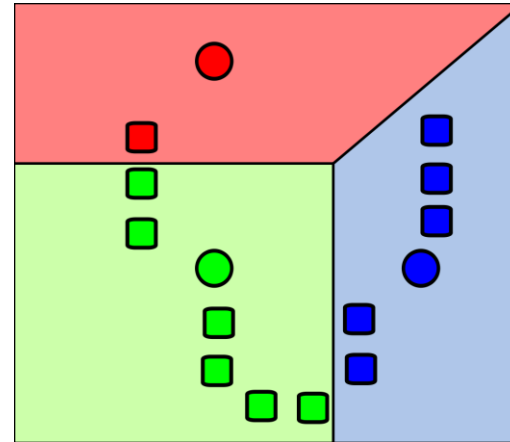
Select the group and find the Mean.

K-Means Clustering

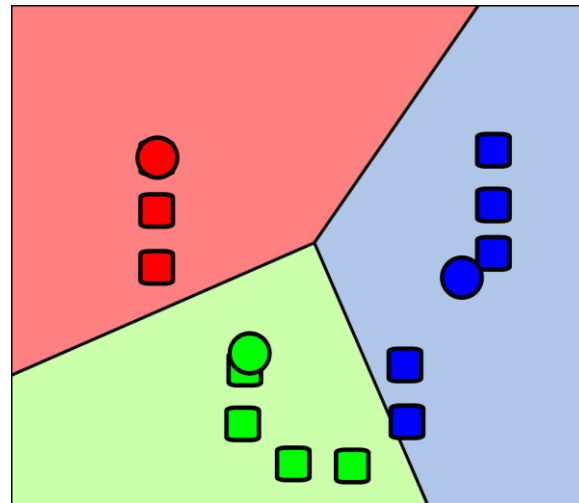
Step-1



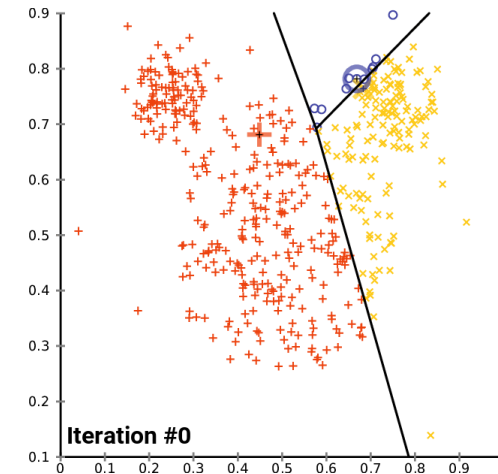
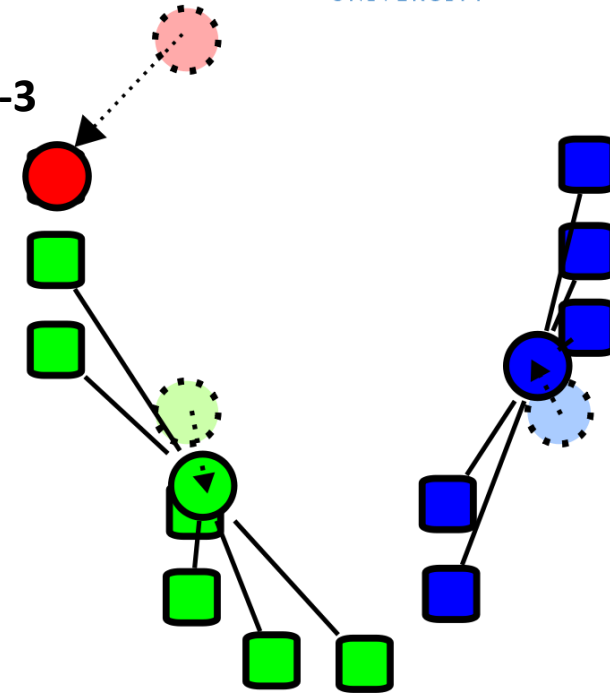
Step-2



Step-4



Step-3



Elbow Method

Elbow Method helps in finding the optimal **k** value. Elbow method represents a graph with the WCSS vs k.

Within-Cluster Sum of Squares(WCSS) - is defined as the sum of square distances between the centroids and each point.

$$WCSS = \sum_{i=1}^k \sum_{x \in C_i} \|x - \mu_i\|^2$$

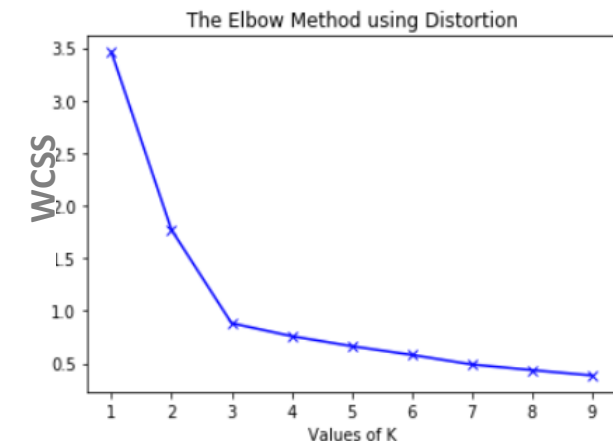
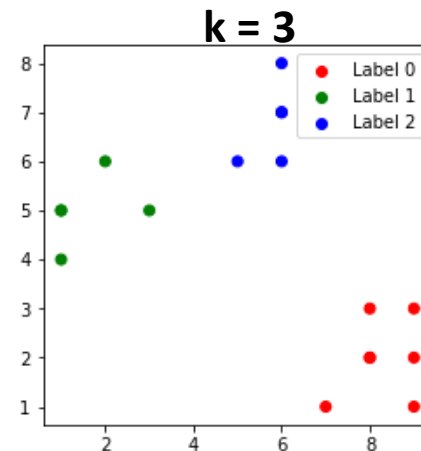
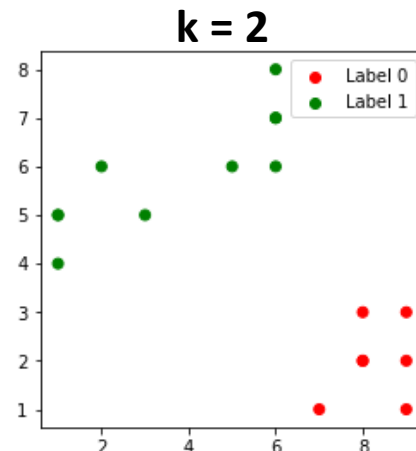
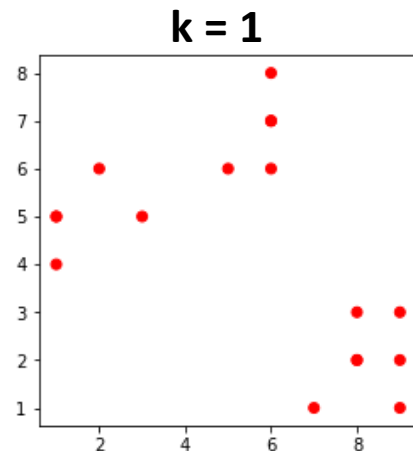
C_i = Cluster i

μ_i = Centroid of Cluster i

x = Data point of Cluster i

Starting with k=2 you can plot a graph with the WCSS vs k.

In each Iteration with a change in K-value the distortion decreases in a linear fashion



K-Means Clustering

Feature learning

k-means clustering has been used as a feature learning step, in either (semi-)supervised learning or unsupervised learning.

The basic approach is first to train a k-means clustering representation, using the input training data (which need not be labelled).

Hierarchical

Hierarchical Clustering

Hierarchical clustering starts by considering each data point as a separate cluster.

The similarity between clusters is typically measured using a distance metric such as Euclidean distance, Manhattan distance, or cosine similarity.

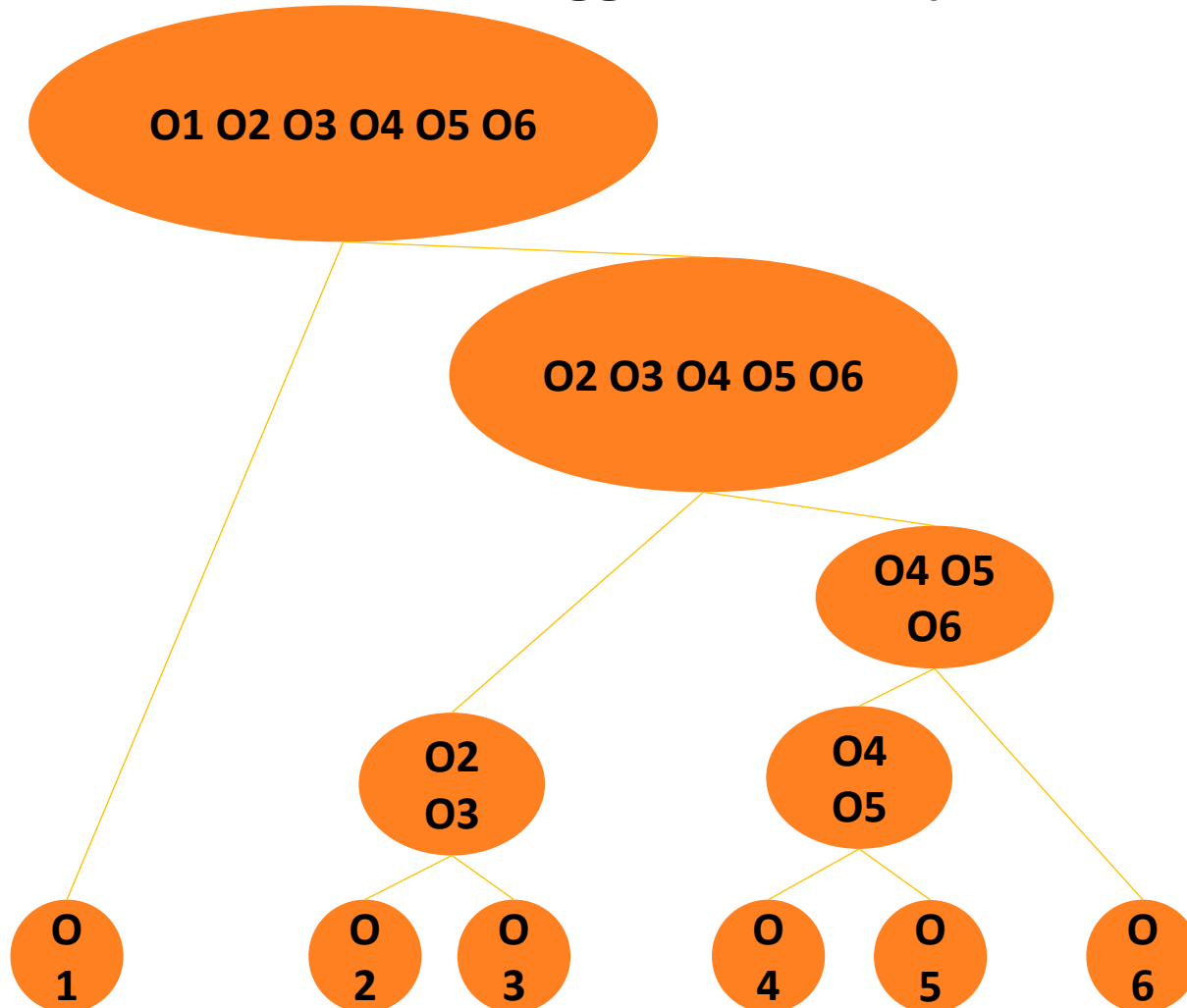
At each step, the algorithm merges or splits clusters based on their similarity.

This clustering technique is of two types:

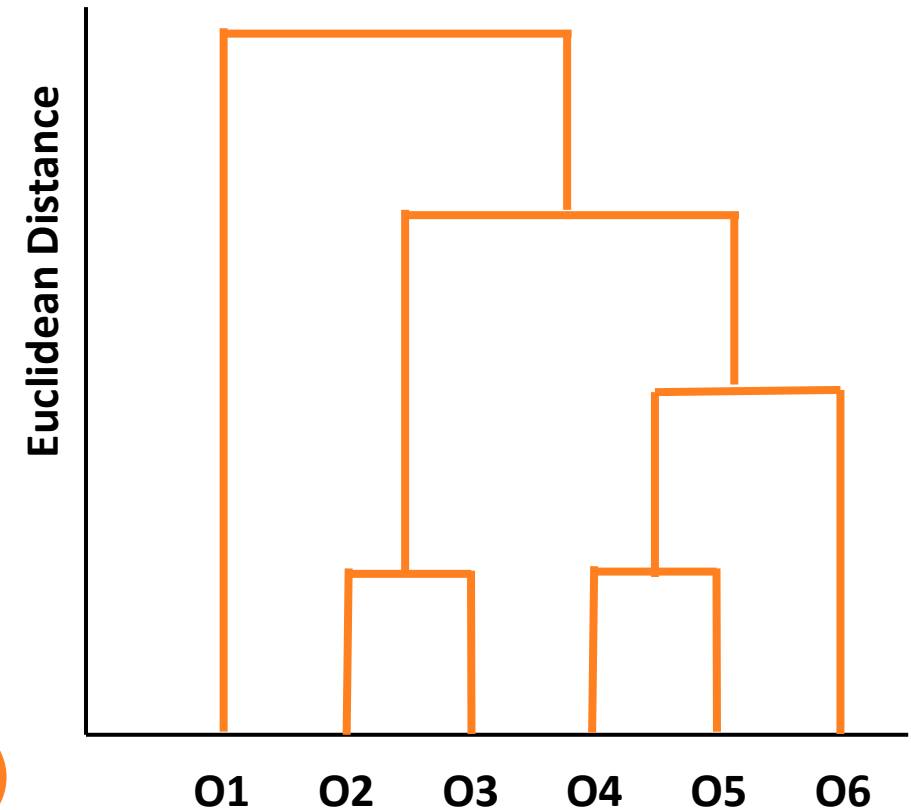
1. **Agglomerative** – Also referred to as **Bottom-Up** technique
2. **Divisive** – Also known as **Top-Down** approach

Hierarchical Clustering – Bottom-Up

Agglomerative (Bottom-Up) Clustering



A Dendrogram is a tree-like diagram that records the sequences of merges or splits.



Hierarchical Clustering – Bottom-Up

Agglomerative (Bottom-Up) Clustering approach begins with each data point as a separate cluster and iteratively merges the closest pairs of clusters until only one cluster remains.

The algorithm proceeds as follows:

1. Compute the proximity matrix
2. Let each data point be a cluster
3. Repeat: Merge the two closest clusters and update the proximity matrix
4. Until only a single cluster remains

Hierarchical Clustering – Top-Down

The **Divisive** (Top-Down) Clustering approach starts with all data points in a single cluster and recursively splits clusters until each data point is in its cluster. The algorithm proceeds as follows:

Compute the proximity matrix: Calculate the pairwise distances/similarities between all data points.

Split the furthest cluster: Identify the cluster with the largest within-cluster distance/similarity and split it into two clusters.

Update the proximity matrix: Recompute the distances/similarities between the new clusters and all other clusters.

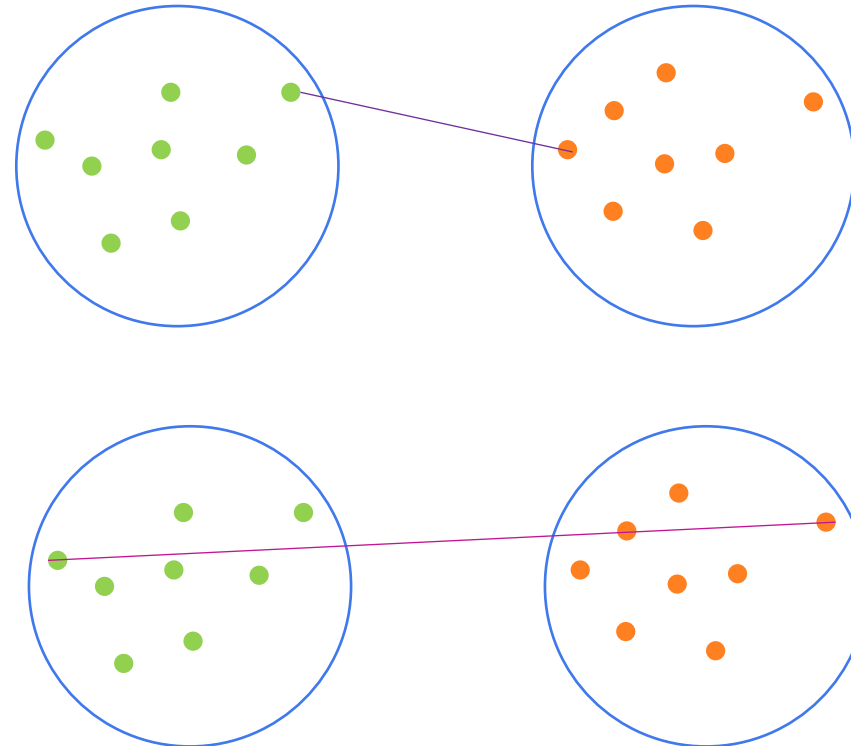
Repeat the process until each data point is in its own cluster.

Similarity Between Clusters

To merge or divide the clusters calculating the similarity between two clusters is important.

There are certain approaches which are used to calculate the similarity between two clusters:

1. MIN
2. MAX
3. Group Average
4. Distance Between Centroids
5. Ward



Hierarchical clustering - Summary

Hierarchical clustering has several advantages, including its ability to visualize the hierarchical structure of the data and its flexibility in producing clusters at different levels of granularity.

However, it can be computationally expensive, especially for large datasets, and the choice of distance metric can significantly affect the resulting clusters.

Overall, hierarchical clustering is a powerful technique for exploring and understanding the structure of complex datasets.

DBSCAN

DBSCAN Algorithm

DBSCAN – Density-Based Spatial **Clustering** of Applications with **Noise**

The Algorithm uses density to cluster datapoints

Density is the number of points present in a unit volume of space. (Both higher & lower Dimensions)



DBSCAN Algorithm

DBSCAN – Density-Based Spatial **Clustering** of Applications with **Noise**

Min samples 'Z' (Min points)

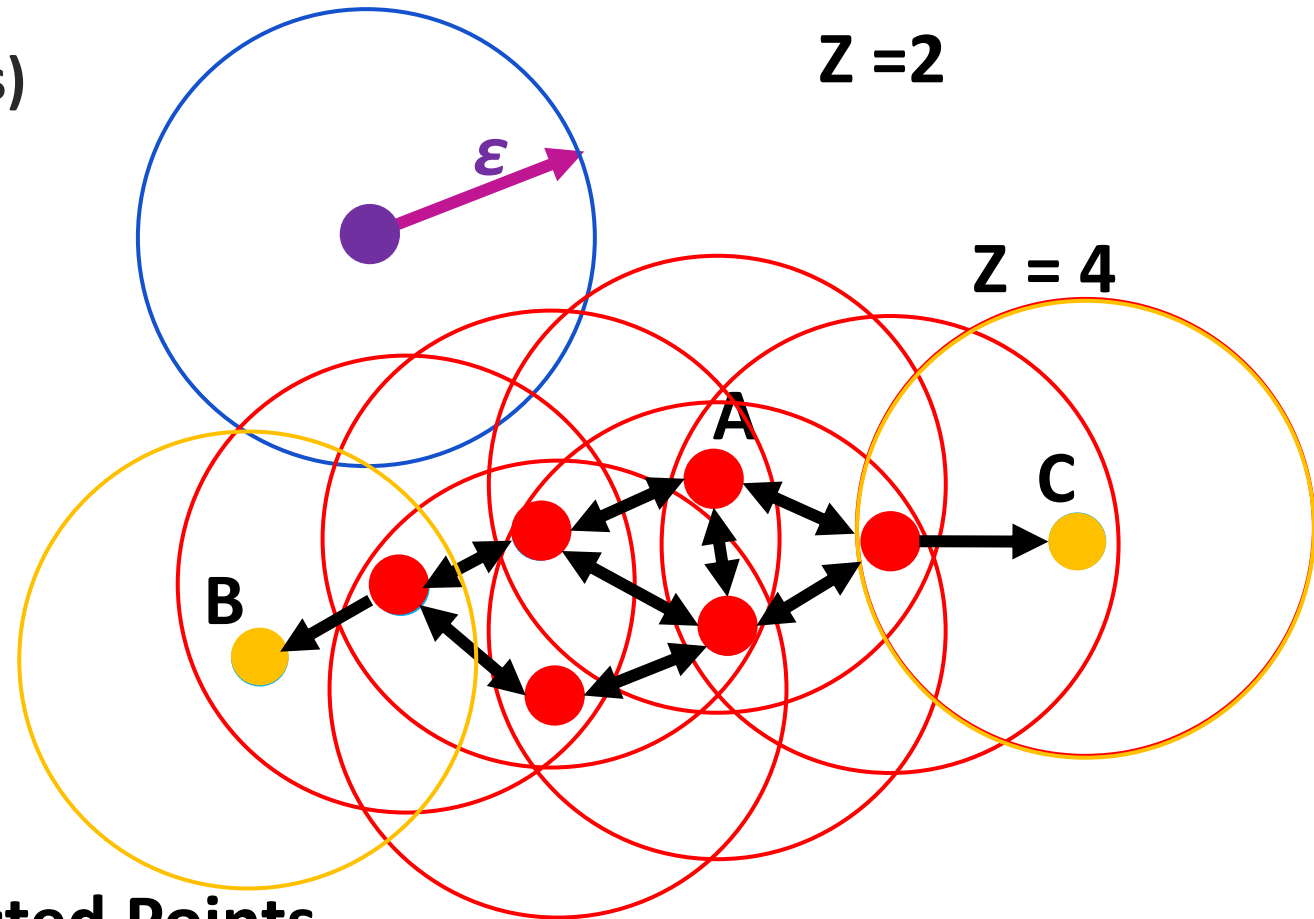
Core Point

Border Point

Noise Point

↔ **Density Edge**

— **Density Connected Points**



Consider a set of points in some space to be clustered.

Let ϵ be a parameter specifying the radius of a neighborhood with respect to some point.

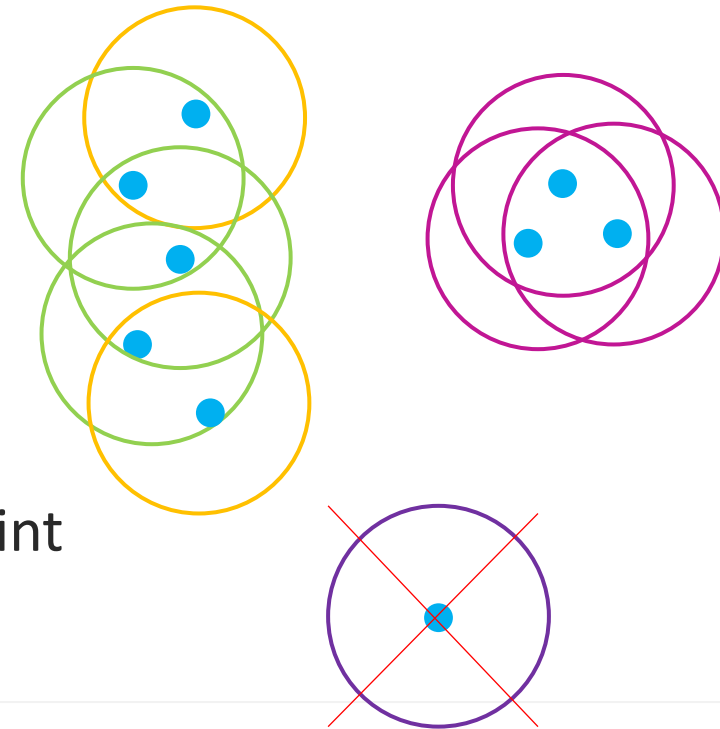
- A point p is a *core point* if at least min sample points are within distance ϵ of it (including p).
- A point q is *directly reachable* from p if point q is within distance ϵ from core point p . Points are only said to be directly reachable from core points.
- A point q is *reachable* from p if there is a path $p_1, \dots, p_n$ with $p_1 = p$ and $p_n = q$, where each p_{i+1} is directly reachable from p_i .
- All points not reachable from any other point are *outliers or noise points*.

DBSCAN Algorithm Steps

DBSCAN – Density-Based Spatial **Clustering** of Applications with **Noise**

STEPS:

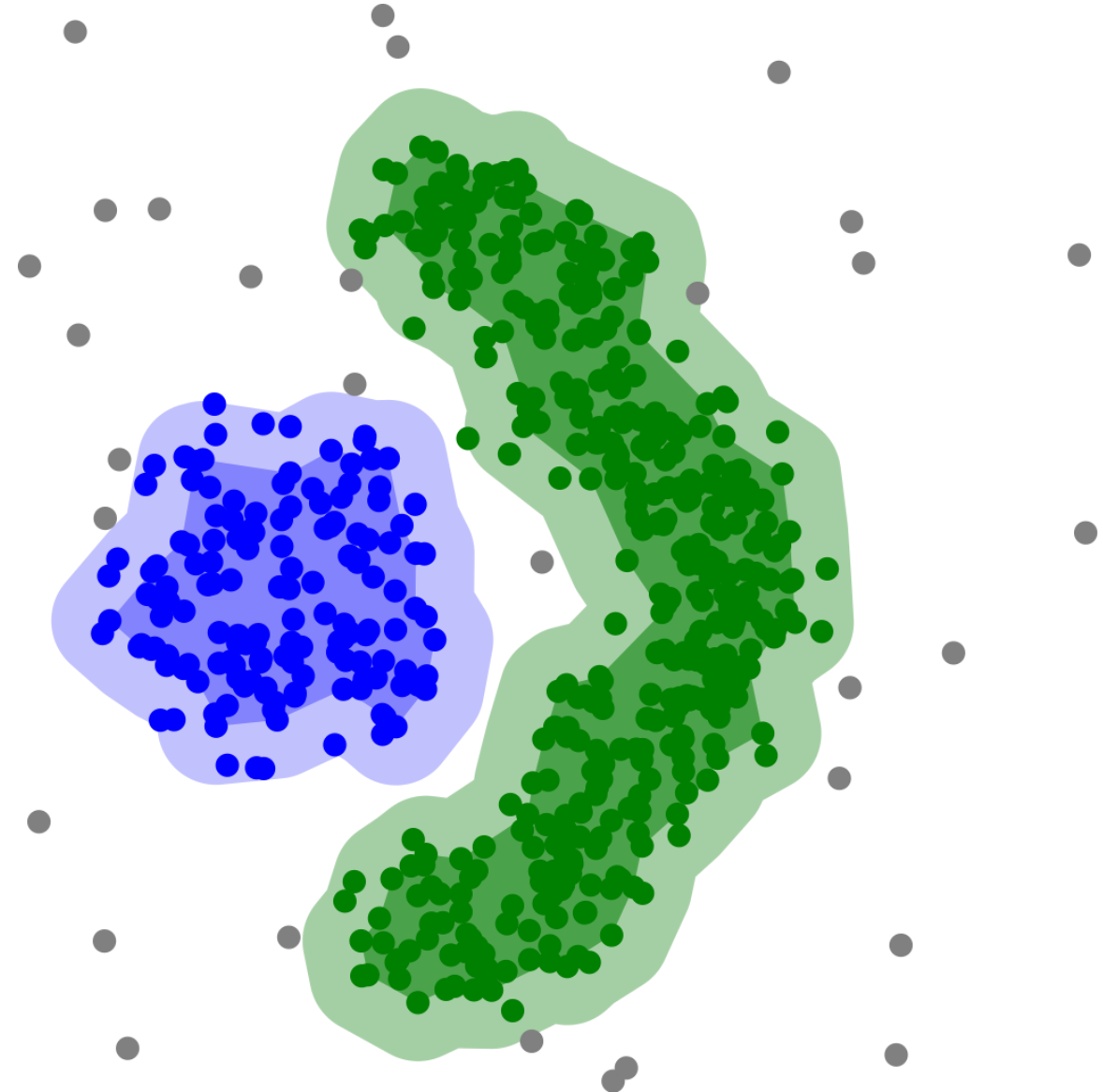
1. Classify the points (core, noise, boundary)
2. Discard the Noise
3. Assign Cluster to a core point
4. Color all the DCP of a core point
5. Repeat steps 3, and 4 for every uncolored core point
6. Color boundary points to the nearest core point



DBSCAN Vs K-Means

DBSCAN can find non-linearly separable clusters.

This dataset cannot be adequately clustered with k-means clustering.



DBSCAN - Advantages

- DBSCAN does not require one to specify the number of clusters in the data as opposed to **k-means**.
- DBSCAN can find arbitrarily-shaped clusters. It can even find a cluster completely surrounded by (but not connected to) a different cluster.
- Due to the **MinPts** parameter, the so-called **single-link effect** (different clusters being connected by a thin line of points) is reduced.
- DBSCAN has a notion of **noise**, and is robust to **outliers**.
- DBSCAN requires just two parameters and is mostly insensitive to the ordering of the points in the database.
- The parameters minPts and ϵ must be set by a domain expert, if the data is well understood.

DBSCAN - Disadvantages

This metric can be rendered almost useless for high-dimensional data due to the so-called "Curse of dimensionality", making it difficult to find an appropriate value for ϵ .

The border points that are reachable from more than one cluster can be part of an either cluster, depending on the data's processing order.

If the data and scale are not well understood, choosing a meaningful distance threshold ϵ can be difficult.

Evaluating Clustering Models

Silhouette Score.

Silhouette refers to a method of interpretation and validation of consistency within clusters of data.

The silhouette value is a measure of how similar an point is to its own cluster (cohesion) compared to other clusters (separation).

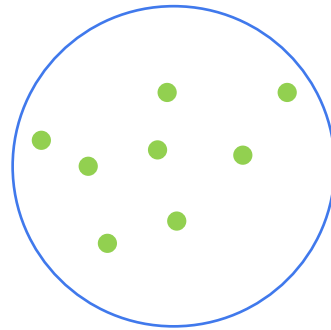
The silhouette ranges from -1 to $+1$, where a high value indicates that the object is well matched to its own cluster and poorly matched to neighboring clusters.

If most points have a high value, then the clustering configuration is appropriate. If many points have a low or negative value, then the clustering configuration may have too many or too few clusters.

Evaluating Clustering Models

Computing Silhouette Score.

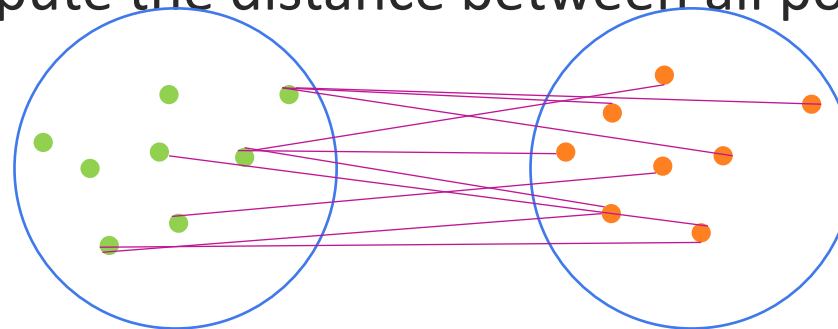
Take one cluster and from centroid calculate the distance to all the datapoints



$$a(i) = \frac{1}{|C_I| - 1} \sum_{j \in C_I, i \neq j} d(i, j)$$

From the nearest second cluster compute the distance between all points of C_1 to all points of C_2 and take average.

$$b(i) = \min_{J \neq I} \frac{1}{|C_J|} \sum_{j \in C_J} d(i, j)$$

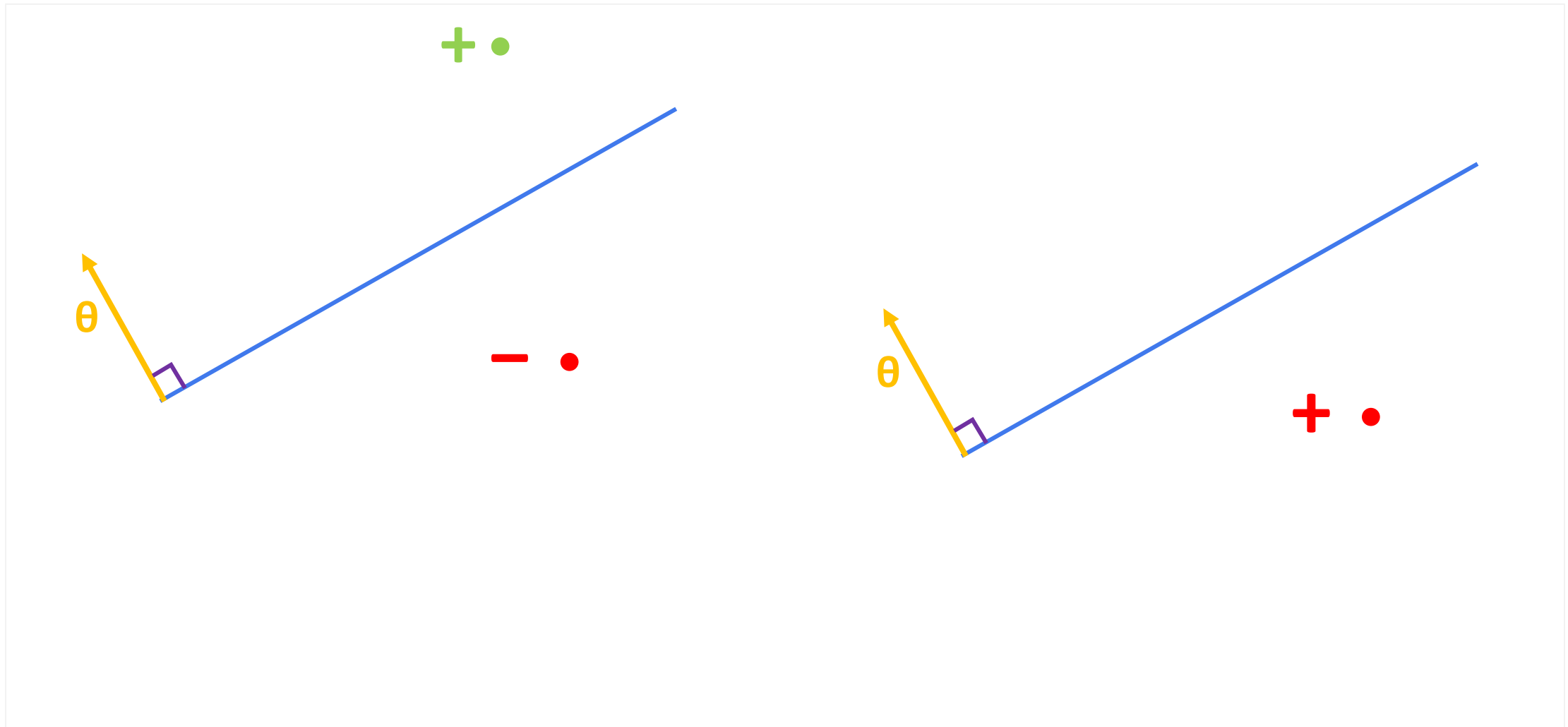


silhouette (value) of one data point i

$$s(i) = \frac{b(i) - a(i)}{\max\{a(i), b(i)\}}, \text{ if } |C_I| > 1$$

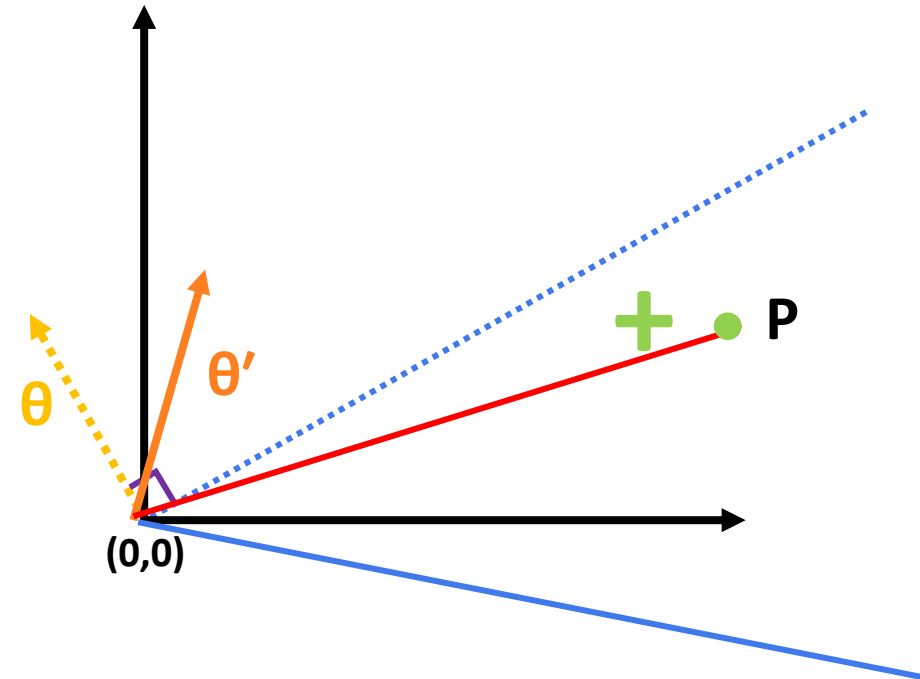
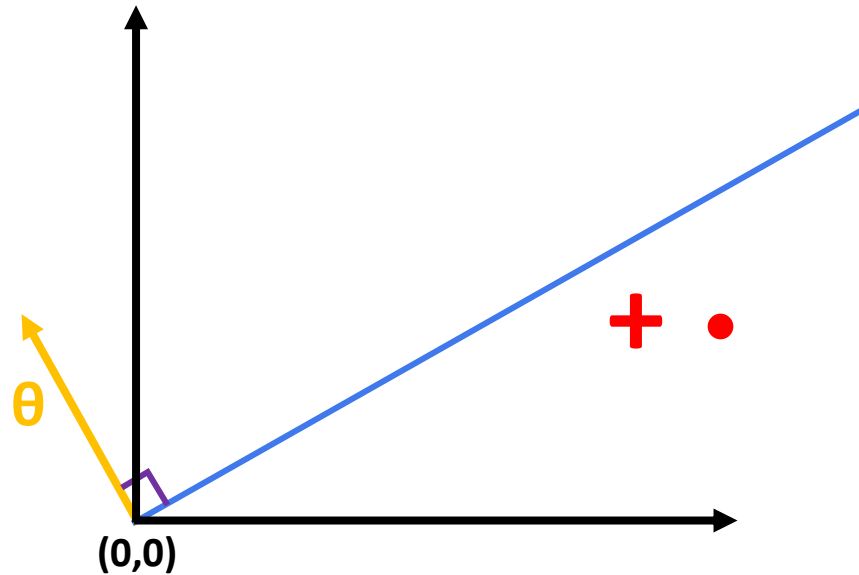
Perceptron

Misclassified Data Points



Perceptron Learning Algorithm

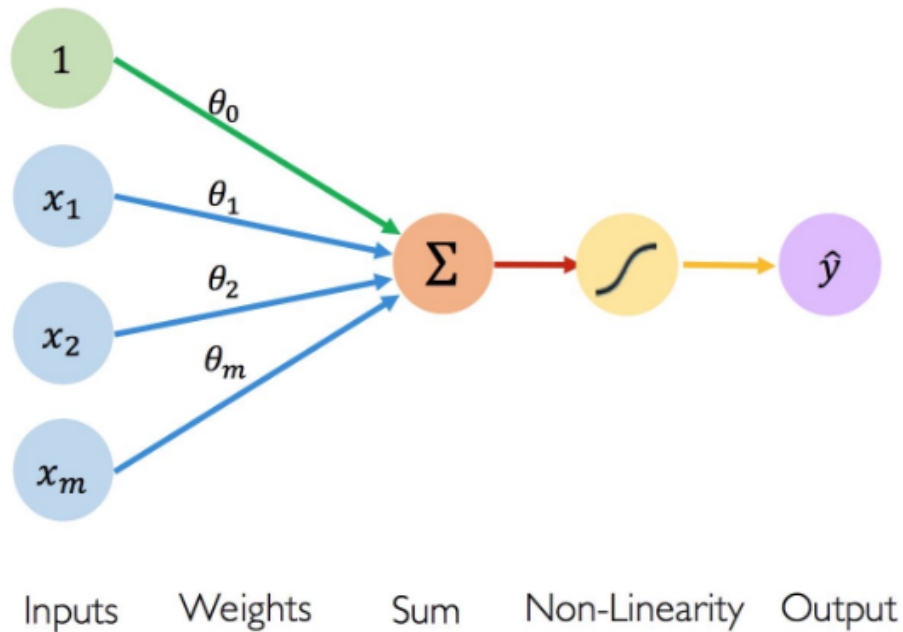
Algorithm is learning one example at a time.



A factor of vector 'P' is added to the parameter vector θ to get the resulting vector θ' . This will shift the decision boundary to new position which can now correctly classify the point 'P'. $\theta' = \theta + P$

A Neuron in Neural Network

A logistic unit, or neuron



Output

Linear combination of inputs

$$\hat{y} = g \left(\theta_0 + \sum_{i=1}^m x_i \theta_i \right)$$

Non-linear activation function

Bias

END

Gaussian Mixture Models

What if data has non-circular clusters?

The spread of data points is non-circular and have different orientations like:

Gaussian Mixture Models are the extension of K-Means models that clusters the data as Gaussian distribution models.

So it is just not computing the centroids by mean would lead to the clusters but also the covariance can explain their ellipsoidal shape

The GMM are based on the likelihood of the observed data

Gaussian Mixture Models

Expectation Maximization (EM) Algorithm